

Chapter 13

Grouped Summaries

by Lorraine Gaudio

Version May 29, 2026

Contents

1	Overview	2
2	Setting Up Your Notes	2
2.1	Create a Quarto File ()	2
2.2	YAML Header ()	3
2.3	Headings ()	3
2.4	Packages	3
3	Counts, Center, and Spread	3
3.1	Data Terminology	4
3.2	Count	4
3.3	Center	4
3.4	Spread	4
3.5	What matters for this chapter?	5
4	dplyr's count()	5
4.1	Count one variable	5
4.2	Count multiple variables	6
5	dplyr's summarize()	6
5.1	count() versus summarize()	7
6	dplyr's group_by()	7
6.1	Explicit vs. Default Naming	8
7	Multiple Statistics	8
8	Multiple Variables	9
8.1	ungroup()	10
9	Summary	11
10	Chapter Terms	12
11	Practice Space	12
11.1	Setup	13
11.2	Task 1	13
11.3	Task 2	14
11.4	Task 3	15
11.5	Task 4	15

11.6 Task 5	16
11.7 Task 6	16

1 Overview

In Chapters 11 and 12, you learned how to subset and transform data with `dplyr` using functions such as `select()`, `filter()`, `mutate()`, and `case_when()`. In this chapter, you will extend that workflow by learning how to organize data into groups and then calculate summaries for those groups. You will use `count()` to make frequency tables, `summarize()` to compute statistics such as mean and standard deviation, and `group_by()` to tell R which subgroups to analyze separately. You will also practice computing multiple statistics at once and working with more than one grouping variable.

This chapter is about learning when and how to ask R for grouped summaries, and then explaining what those summaries mean. At the same time, you will continue building reproducible workflow habits in Quarto by using headings, rendering often, checking your output, and memoing what your code is doing and what you learned from it.

By the end of Chapter 13 you will be able to:

- Use `count()` to create frequency tables for one categorical variable.
- Use `count()` with more than one variable to count combinations of categories.
- Use `summarize()` to calculate summary statistics such as mean, standard deviation, maximum, and row count.
- Use `group_by()` to compute separate results for each group instead of one result for the whole dataset.
- Explain how grouping changes the meaning of a summary result.
- Combine `group_by()` and `summarize()` in a pipeline to calculate grouped summaries.
- Compute multiple summary statistics in a single `summarize()` call.
- Distinguish between a grouping variable and a summary variable when analyzing data.
- Choose when `count()` is the better tool and when `summarize(n = n())` is the better tool.
- Remove grouping after summarizing by using `.groups = "drop"` or `ungroup()`.
- Continue using Quarto to organize code, output, checks, and written interpretation in a reproducible analysis document.

Keep these goals in mind as you move through each section.

2 Setting Up Your Notes

In Chapter 11, you learned how to create a Quarto file, keep the YAML header, use Markdown headings, write memos outside code chunks, and control chunks with `#|` options. In Chapter 12, you add a table of contents and section numbering, and use inline R code inside your sentences. In Chapter 13, you will practice using these skills.

2.1 Create a Quarto File ()

For those of you who **have GitHub** and a course project, navigate to your course folder and open your course project. This will open RStudio. Click “Pull” in the Git pane to get the latest version of the course materials. After completing work, save, stage, commit, and push.

If you **don’t have GitHub** or a course project, just open RStudio and set your working directory to your course folder.

Open a new Quarto document in your course folder and save it as: `chapter13_notes.qmd` (File > New File > Quarto Document).

2.2 YAML Header ()

Copy and paste the YAML header from this file into your new Quarto document. Update the title, author, date, and team fields in the YAML header.

```
---
title: "Chapter 13 Notes"
subtitle: "group_by() and summarize()"
author: "YOUR NAME"
date: today
toc: true
toc-depth: 2
number-sections: true
format:
  html:
    code-fold: false
---
```

2.3 Headings ()

Use the section headings in this file to create the structure of your notes. You can copy and paste the section headings into your new Quarto document.

```
# Loading Packages

# Counts, Center, and Spread

# count()

# summarize()

# group_by()

# multiple statistics

# multiple variables
```

Work through the sections under each heading. Render to HTML often to see the results.

2.4 Packages

Install once (if needed): `install.packages("dplyr"); install.packages("dslabs")`

Load the packages

```
#
library(dslabs) # Data science labs package
library(dplyr)  # Data manipulation package
```

3 Counts, Center, and Spread

Before using `summarize()` and `count()`, it helps to review a few data words you have likely seen before. You do **not** need to calculate these by hand in this chapter. You do need to understand what they mean so you

can choose the right function and explain your results.

3.1 Data Terminology

A **numeric variable** stores numbers that can be measured or calculated, such as `mpg`, height, or test score.

A **categorical variable** stores labels or group names, such as state, disease, or cylinder count.

In this chapter, we often **group by a categorical variable** and then **summarize a numeric variable**.

3.2 Count

A **count** tells you how many observations there are.

- For a whole dataset, count means the total number of rows.
- For grouped data, count means the number of rows in each group.

If you want to know how many cars have 4 cylinders, 6 cylinders, or 8 cylinders, you are asking for a **count**.

3.3 Center

Center describes a typical value.

3.3.1 Mean

The **mean** is the average.

It adds the values and divides by how many values there are.

If the mean `mpg` for cars with 4 cylinders is higher than the mean `mpg` for cars with 8 cylinders, then the 4-cylinder cars are more fuel efficient on average.

3.3.2 Median

The **median** is the middle value when the data are sorted.

Median is useful when extreme values might pull the mean too high or too low.

3.4 Spread

Spread describes how much the values vary.

3.4.1 Standard Deviation

The **standard deviation** tells you how far values tend to be from the mean.

- A **small** standard deviation means the values are packed closely together.
- A **large** standard deviation means the values are more spread out.

If one cylinder group has a much larger standard deviation for `mpg`, that group has more variability in fuel efficiency.

3.4.2 Range

The **range** is the distance from the smallest value to the largest value.

Range gives a quick sense of spread, but it only uses the minimum and maximum values.

3.5 What matters for this chapter?

In Chapter 13, the most important ideas are:

- use **count** when you want to know how many rows are in a dataset or group
- use **mean** when you want an average
- use **standard deviation** when you want to describe spread
- use a **categorical variable** to make groups
- use a **numeric variable** when calculating statistics like mean or standard deviation

4 dplyr's count()

You can use `count()` to quickly count rows for each category or combination of categories.

`count()` is especially useful when you want a **frequency table**.

?count

The SYNTAX

```
data %>% count(variable_name)
```

Or, for more than one grouping variable:

```
data %>% count(variable1, variable2, ...)
```

4.1 Count one variable

Count how many cars have each cylinder value.

```
#  
mtcars %>%  
  count(cyl)
```

```
##   cyl  n  
## 1    4 11  
## 2    6  7  
## 3    8 14
```

The code above counts how many rows fall into each value of `cyl`. The result is a new table with one row per cylinder group and a column named `n` that stores the counts.

This is useful when you want a quick summary of how many observations are in each category.

4.1.1 count() versus length()

`length()` counts how many values are in a vector.

`count()` counts rows in a data frame for each category or combination of categories.

4.1.2 count() with sorting

Sometimes you want to see the biggest groups first.

Count cars by cylinder and sort from largest count to smallest count.

```
#  
mtcars %>%  
  count(cyl, sort = TRUE)
```

```
##   cyl  n
## 1   8 14
## 2   4 11
## 3   6  7
```

`sort = TRUE` arranges the results so the largest counts appear first.

4.2 Count multiple variables

You can also count combinations of categories.

Count how many cars fall into each combination of cylinder count and transmission type.

```
#
mtcars %>%
  count(cyl, am)
```

```
##   cyl am  n
## 1   4  0  3
## 2   4  1  8
## 3   6  0  4
## 4   6  1  3
## 5   8  0 12
## 6   8  1  2
```

`count(cyl, am)` creates one row for each unique combination of `cyl` and `am`, then counts how many rows are in each combination.

This is useful when one grouping variable is not enough. However, you may want to determine more than just counts for each group. In that case, you can use `group_by()` and `summarize()` instead.

5 dplyr's summarize()

You can use `summarize()` to compute summary statistics like the mean, median, etc.

```
?summarize
```

The SYNTAX data %>% `summarize(new_column_name = some_operation(column_name))`

Find the overall mean mpg for all cars.

```
#
mtcars %>%
  summarize(mean_mpg = mean(mpg))
```

```
##   mean_mpg
## 1 20.09062
```

The code above creates a new column called `mean_mpg` that contains the mean of the `mpg` column from the `mtcars` dataset.

We name the new column using `new_name = calculation`.

Find several summary statistics for mpg: mean, max, and count of cars.

```
#
mtcars %>%
  summarize(mean_mpg = mean(mpg),
            max_mpg = max(mpg),
            count = n())
```

```
##   mean_mpg max_mpg count
## 1 20.09062   33.9    32
```

Notice: There is not piping `%>%` inside of the `summarize()` function. Instead, a comma `,` is used between the `mean()`, `max()` and `n()` calculations.

`summary()` is a base R function that gives a quick overall description of an object, such as basic statistics for each variable in a data frame. `summarize()` from `dplyr` is used to create your own summary results, usually after `group_by()`, such as the mean or count for each group.

5.1 `count()` versus `summarize()`

You can often get the same result in two ways.

Use `group_by()` and `summarize()` to produce the same counts by cylinder.

```
#
mtcars %>%
  group_by(cyl) %>%
  summarize(n = n())
```

```
## # A tibble: 3 x 2
##   cyl     n
##   <dbl> <int>
## 1     4    11
## 2     6     7
## 3     8    14
```

This gives the same idea as `count(cyl)`. The difference is that `count()` is shorter and faster to write when your main goal is just to count rows.

Use `count()` when you mainly want frequencies. Use `group_by()` with `summarize()` when you want to calculate other statistics too, such as mean and standard deviation.

6 `dplyr`'s `group_by()`

`group_by()` changes the behavior of summary functions.

The SYNTAX

```
data %>% group_by(variable1, variable2, ...) %>% some_operation()
```

Find the mean mpg for each cylinder count.

```
#
cyl_means <- mtcars %>%
  group_by(cyl) %>%
  summarize(mean_mpg = mean(mpg))
```

`group_by(cyl)` splits the cars into groups based on cylinder count, and `summarize()` calculates `mean(mpg)` for each group. `mean_mpg` is the name of the new summary column.

Notice: that we chain `group_by()` and `summarize()` together using the pipe operator `%>%`. This allows us to first group the data by the specified variable(s) and then apply the summary function to each group.

```
#
cyl_means
```

```
## # A tibble: 3 x 2
##   cyl mean_mpg
```

```
##    <dbl>    <dbl>
## 1      4     26.7
## 2      6     19.7
## 3      8     15.1
```

Grouping cars by `cyl` is useful because cars with different numbers of cylinders often have different fuel efficiency, so one overall average mpg would hide those differences. By grouping first, you can compare the typical mpg for 4-, 6-, and 8-cylinder cars separately.

6.1 Explicit vs. Default Naming

If you do not give the summary column a name, **dplyr** uses the summary expression itself as the column name by default. So `summarize(mean(mpg))` creates a column named `mean(mpg)` instead of a cleaner name like `mean_mpg`.

Find the mean mpg for each cylinder count without naming the summary column.

```
#
mtcars %>%
  group_by(cyl) %>%
  summarize(mean(mpg))
```

```
## # A tibble: 3 x 2
##   cyl `mean(mpg)`
##   <dbl>         <dbl>
## 1     4         26.7
## 2     6         19.7
## 3     8         15.1
```

The default name is the code itself. Default names can be confusing when sharing results. Naming columns makes tables clearer. Use it whenever you share results.

7 Multiple Statistics

We can create several summary columns in one call, separated by commas.

?n to learn about the `n()` function.

```
#
cyl_stats <- mtcars %>%
  group_by(cyl) %>%
  summarize(mean_mpg = mean(mpg),
            sd_mpg    = sd(mpg),
            n         = n())
#
cyl_stats
```

```
## # A tibble: 3 x 4
##   cyl mean_mpg sd_mpg    n
##   <dbl>   <dbl> <dbl> <int>
## 1     4    26.7  4.51    11
## 2     6    19.7  1.45     7
## 3     8    15.1  2.56    14
```

`n()` counts rows in each group. This is similar to the function `length()` which counts the number of values in a vector or row in a data set. It is different from `length` because `n()` works within `summarize()` and counts the number of rows in each group created by `group_by()`. In this way, `n()` is like `count()`, but it is used within `summarize()`. Using `n()` is a quick way to see how many observations are in each group.

8 Multiple Variables

`group_by` one column creates one group for each unique value. `group_by` two columns creates one group for each unique combination.

How many groups will `group_by(cyl, am)` create?

Predict how many groups will be created by `group_by(cyl, am)` before running the code.

```
# Pre-check
# Recall that mtcars$cyl has three unique values:
unique(mtcars$cyl)
```

```
## [1] 6 4 8
```

```
# Recall that mtcars$am has two unique values:
unique(mtcars$am)
```

```
## [1] 1 0
```

Two variables with 3 and 2 unique values creates $3 \times 2 = 6$ groups.

Find the mean mpg for each combination of cylinder count and transmission type.

```
#
mtcars %>%
  group_by(cyl, am) %>%
  summarize(avg_mpg = mean(mpg),
            count = n())
```

```
## `summarise()` has regrouped the output.
## i Summaries were computed grouped by cyl and am.
## i Output is grouped by cyl.
## i Use `summarise(.groups = "drop_last")` to silence this message.
## i Use `summarise(.by = c(cyl, am))` for per-operation grouping
##   (`?dplyr::dplyr_by`) instead.

## # A tibble: 6 x 4
## # Groups:   cyl [3]
##   cyl    am avg_mpg count
##   <dbl> <dbl>   <dbl> <int>
## 1     4     0    22.9     3
## 2     4     1    28.1     8
## 3     6     0    19.1     4
## 4     6     1    20.6     3
## 5     8     0    15.0    12
## 6     8     1    15.4     2
```

Comment (example): - Group 1: cyl = 4, am = 0 (4-cylinder automatic cars)

- Group 2: cyl = 4, am = 1 (4-cylinder manual cars)
- Group 3: cyl = 6, am = 0 (6-cylinder automatic cars)
- Group 4: cyl = 6, am = 1 (6-cylinder manual cars)
- Group 5: cyl = 8, am = 0 (8-cylinder automatic cars)
- Group 6: cyl = 8, am = 1 (8-cylinder manual cars)

8.1 ungroup()

With two grouping vars, the default keeps the earlier group(s). With `.groups = "drop"` (or `ungroup()`), the result is not grouped.

A. Default behavior: keeps earlier groups

Show that with two groups (`cyl, gear`), `summarize()` keeps the earlier group (`cyl`) by default.

```
#  
# A) Default behavior: "drop_last" (keeps earlier groups)  
cyl_gear_stats <- mtcars %>%  
  group_by(cyl, gear) %>%  
  summarize(mean_mpg = mean(mpg)) # no .groups set  
  
## `summarise()` has regrouped the output.  
## i Summaries were computed grouped by cyl and gear.  
## i Output is grouped by cyl.  
## i Use `summarise(.groups = "drop_last")` to silence this message.  
## i Use `summarise(.by = c(cyl, gear))` for per-operation grouping  
##   (`?dplyr::dplyr_by`) instead.  
  
#  
group_vars(cyl_gear_stats) # what groups remain?  
  
## [1] "cyl"  
  
# Verify  
is_grouped_df(cyl_gear_stats) # expect TRUE
```

```
## [1] TRUE
```

The Default = `drop_last` from (`cyl,gear`) down to (`cyl`).

You'll want to be explicit when you don't want any grouping left.

B. Make it explicit inside `summarize()`

Return a plain, ungrouped table by telling `summarize()` to drop all groups.

```
#  
# B) Make it explicit: drop *all* groups in the result  
cyl_gear_stats_drop <- mtcars %>%  
  group_by(cyl, gear) %>%  
  summarize(mean_mpg = mean(mpg), .groups = "drop")  
  
#  
group_vars(cyl_gear_stats_drop) # expect character(0)  
  
## character(0)  
  
#  
is_grouped_df(cyl_gear_stats_drop) # expect FALSE
```

```
## [1] FALSE
```

Using `.groups = "drop"` tells `summarize()` to remove all grouping after the summary operation, resulting in a plain data frame without any grouping structure.

C. Alternative: drop groups after summarizing

Achieve the same ungrouped result using `ungroup()` after `summarize()`

```
#
# C) Alternative: drop groups after summarizing
cyl_gear_stats_ungroup <- mtcars %>%
  group_by(cyl, gear) %>%
  summarize(mean_mpg = mean(mpg)) %>%
  ungroup()

## `summarise()` has regrouped the output.
## i Summaries were computed grouped by cyl and gear.
## i Output is grouped by cyl.
## i Use `summarise(.groups = "drop_last")` to silence this message.
## i Use `summarise(.by = c(cyl, gear))` for per-operation grouping
##   (`?dplyr::dplyr_by`) instead.

#
group_vars(cyl_gear_stats_ungroup) # expect character(0)

## character(0)

# Verify
is_grouped_df(cyl_gear_stats_ungroup) # expect FALSE

## [1] FALSE
```

Using `ungroup()` after `summarize()` removes all grouping from the resulting data frame, similar to using `.groups = "drop"` within `summarize()`. Both methods achieve the same outcome of returning an ungrouped data frame.

Pick either B) or C) based on your coding style preference.

9 Summary

In this chapter, you learned how to describe data by groups using `dplyr`. You reviewed the basic ideas of count, center, and spread so you could interpret grouped results, then used `count()` to make frequency tables, `summarize()` to compute statistics, and `group_by()` to split a dataset into subgroups before summarizing. You also practiced calculating multiple statistics in one call, using `n()` inside `summarize()` to count rows within groups, and working with combinations of grouping variables. Most importantly, you learned that grouping changes what a summary means: instead of one result for the whole dataset, you can produce separate results for each category or combination of categories. This is a core skill for later data analysis because analysts often need to compare subgroups, not just describe an entire dataset with one number.

Function	Purpose
<code>group_by()</code>	Changes the behavior of summary functions by creating groups based on discrete variables.
<code>summarize()</code>	Computes summary statistics for each group created by <code>group_by()</code> .
<code>count()</code>	Counts the number of occurrences of each unique value in a column, often used for categorical variables.
<code>n()</code>	Counts the number of rows in the current group when used within <code>summarize()</code> .
<code>length()</code>	Counts the number of values in a vector or rows in a data set.
<code>unique()</code>	shows the distinct values
<code>sum()</code>	adds up values in a vector

Function	Purpose
<code>max()</code>	finds the largest value in a vector

10 Chapter Terms

Categorical Data: Data whose values place observations into groups or categories. The categories may have no natural order (nominal) or may have an order with uneven gaps between categories (ordinal). In R, categorical data are often stored as factors.

Center: The middle or typical location of a dataset. Measures of center, such as the mean and median, summarize where values tend to cluster.

Count: The number of observations or values in a dataset, group, or category. In introductory statistics, count tells you how many cases are present.

Frequency: The number of times a value or category appears in a dataset.

Mean: The arithmetic average of a set of numbers, found by adding all values and dividing by the number of values.

Median: The middle value in an ordered dataset. If there is an even number of values, the median is the average of the two middle values.

Numerical Data: Data recorded as numbers, where the values have meaningful magnitudes and consistent intervals. Numerical data are also called *quantitative* data.

Pipe: An operator that passes the result of one step directly into the next step, letting you write code as a readable sequence of actions instead of nesting functions inside each other. In tidyverse code, the pipe is usually `%>`, so `x %> f()` means “take `x`, then apply `f()`”; in newer base R, a similar built-in pipe is `|>`.

Range: A simple measure of spread found by subtracting the smallest value from the largest value in a dataset.

Spread: How much the values in a dataset vary or scatter rather than cluster tightly together. Common measures of spread include range, interquartile range (IQR), variance, and standard deviation.

Standard Deviation: A measure of spread that describes how far values typically vary from the mean. A small standard deviation means values are clustered close to the mean, while a large standard deviation means they are more spread out.

11 Practice Space

This Practice Space helps you turn Chapter 13 concepts into a reproducible analysis workflow.

In this chapter, you practiced using `group_by()`, `summarize()`, `count()`, and `n()` to describe data. You worked with grouped summaries, multiple statistics, and combinations of categorical variables.

This work matters because analysts often need to answer questions such as:

- How many observations are in each group?
- What is the average value within each group?
- How much do values vary within each group?
- What happens when we group by more than one variable?

These tasks give you practice writing code, checking your output, and explaining what your results mean in words. That is part of doing reproducible work that someone else can follow and re-run later.

11.1 Setup

1. Create a new Quarto file in your course folder and save it as:
 - `chapter13_practice.qmd`
 - Test Render your document often to see how your Practice Space report is coming together.
 - Submit `chapter13_practice.html`
2. Use the heading structure taught earlier in the chapter. Your Quarto setup for Chapter 13 should include headings, section numbering, and a table of contents, and you should render often to check both formatting and errors.
3. Library it up!
 - Make sure there is script in your document that loads `dplyr` and `dslabs` packages so their functions / datasets load.
4. Memo your work with the `//` /
 - a memo before the code chunk that states the goal of the code and what you are trying to learn or understand
 - the code chunk itself
 - any needed checks after the code
 - a memo after the code chunk that explains what you learned, discovered, or confirmed
 - answers to all comment questions included in the task

If you used help while working, document it in a memo. For example, if you used an LLM or another outside resource to understand an error or concept, briefly disclose that use and explain what help you received. Type all code yourself. This is part of the course expectation for AI-supported learning and memo-based workflow.

Follow this template for each coding task.

What is the goal of the code? What am I trying to learn, check, or demonstrate?

```
#Code
your_code <- "here"

# Include Checks after and pre-checks as needed
colnames(your_code)
head(your_code)
table(your_code)
class(your_code)
unique(your_code)
```

/ What I learned / Next step. 1–2 sentences.

11.2 Task 1

Single grouping

Using `stars`, compute `mean_temperature` **AND** `mean_magnitude` for each type. Name the result `twinkle_twinkle`.

```
data(stars)
?stars
colnames(stars)
```

Restate the Goal. What are you trying to learn, check, or demonstrate?

```
# Code
twinkle_twinkle <- stars %>%
  ____ (____) %>%
  summarize(
    mean_temperature = ____ (____),
    mean_magnitude   = ____ (____),
    .groups = "drop"
  )
```

```
# Checks
twinkle_twinkle
```

Comment on the output.

11.3 Task 2

Single grouping + three stats

Pick one numeric fatty acid in `olive` such as `palmitic`, `oleic`, or another fatty-acid variable. Group by `region` then calculate

- the mean of your chosen fatty acid,
- the standard deviation of your chosen fatty acid, and
- the number of rows in each region.

Store as `Olive_Garden`.

```
data(olive)
?olive
colnames(olive)
```

Restate the goal. What are you trying to learn, check, or demonstrate?

```
# Code
fatty_acid <- "_____"

Olive_Garden <- olive ___
  group_by(region) ___
  summarize(
    mean_value = mean(fatty_acid)___
    sd_value   = sd(fatty_acid)___
    n          = n()
  )
___
```

```
# Checks
Olive_Garden
```

In a short comment, note the region with the highest mean value for the fatty acid you selected.

Why is `region` the grouping variable and the fatty acid column the summary variable?

11.4 Task 3

Counting unique values

Which function can you use to count unique values in a column?

For `us_contagious_diseases` from `dslabs`, determine the number of unique diseases in the column `disease`.

```
data("us_contagious_diseases")
?us_contagious_diseases

# Pre-check
## What are the unique diseases
unique(us_contagious_diseases$disease)

## How many unique diseases are there?
____(unique(us_contagious_diseases$disease))
```

Inline memo sentence

Write this as normal text in your Quarto document, not as code. Replace the blank with the correct function to count unique diseases.

There are ``r ____``(`unique(us_contagious_diseases$disease)``) unique diseases within the ``disease`` column

For `us_contagious_diseases`, determine the number of states in the column `state`.

```
# Pre-check
## What are the unique state
unique(us_contagious_diseases$state)

## How many unique diseases are there?
____(unique(us_contagious_diseases$state))
```

Inline memo sentence

Write this as normal text in your Quarto document, not as code. Replace the blank with the correct function to count unique states.

There are ``r ____``(`unique(us_contagious_diseases$state)``) unique states within the ``state`` column of t

11.5 Task 4

Make and test a prediction

How many rows would you expect if each state has 6 diseases?

11.5.1 Step 1

If there are 51 states in the dataset, and you assume every state has data for all 7 diseases, how many groups will `group_by(state, disease)` create?

- A. $51 + 7 = 58$ groups
- B. $51 \times 7 = 357$ groups
- C. $\frac{51}{7} \approx 7$ groups
- D. $51 - 7 = 44$ groups
- E. 51^7 groups

Explain your answer choice.

11.5.2 Step 2

Test it.

Use `group_by()` both `state` and `disease` and `count()` to count the number of rows for each state and disease combination. Store as `categorical_counts`.

Restate the Goal. What are you trying to learn, check, or demonstrate?

```
# Code
categorical_counts <- us_contagious_diseases %>%
  _____(state, disease) %>%
  _____()

# Check row count
_____(categorical_counts)
```

Comment about the results. Was your prediction of group size correct?

Explain why `group_by()` is useful when reporting the **frequency** of categorical data like `state` and `disease`.

11.6 Task 5

National mean counts by disease

From `us_contagious_diseases`, calculate the national mean of `count` for each `disease`. Then keep only the three diseases with the highest national mean counts. Store as `Infectious_Burden`.

Tip: Arrange the diseases by their national mean and keep the top three.

```
# National means per disease
_____ <- _____ %>%
  group_by(_____) %>%
  summarize(
    _____ = mean(_____, na.rm = TRUE),
    .groups = "drop"
  ) %>%
  arrange(desc(_____))

# Checks
head(Infectious_Burden, 3)
```

Comment on the results. Which three diseases had the highest national mean counts?

11.7 Task 6

Reflection

Write a short paragraph explaining why `group_by()` needs to come before `summarize()` when you want separate results for each group.

Your paragraph should explain the difference between:

- summarizing the whole dataset
- summarizing within groups